Promoting `discovery_okuda` into `src/plexus`

the 3D vertex model becomes part of the language, and what had to be true for that

1. What this is

`discovery_okuda` held the 3D active vertex model, reaction–diffusion on the cell graph, the junction and medioapical myosin pools, and the whole basement-membrane / extracellular-matrix stack: 52 registered operator names across 21 files, and behind them `log/okuda` and `log/okuda_ECM`. None of it was in the framework `paper/plexus2.tex` describes. The core's own `src/plexus/operators/` carried 43 names in 45 one-operator files, and not one mesh or chemistry operator among them.

This note is the record of moving that vocabulary into the core, written as each phase landed rather than summarised afterwards. Its structure follows the paper's four parts — mechanism, equations and symbols, the Plexus2 container, gates — with one section per phase, and its gate tables are generated from the run record by `tools/promotion_note.py` rather than transcribed.

The two things that had to be true. A promotion of this size is only checkable if it can be shown to change nothing. Two gates run on every phase.

1. **The twin run.** The spec is run in okuda *first*, and its twin is run in the codebase; previous okuda results are never the reference, because okuda is still moving. Both sides are submitted *together*, as two `bsub` jobs on `gpu_l4`, and the comparison is array by array, bit for bit, reduced to one sha1 per side.
2. **The okuda_ECM gates 00–04**, each assigned to the phase that enables it, each with a table of thresholds declared *before* the promoted run. §5.

2. Phase 0 — the engine

2.1 The half-edge mesh becomes an engine concept

A closed cellular surface is stored as a flat half-edge table: three parallel `int64` arrays `E_srce`, `E_trgt`, `E_face`, plus the counts `nF` (faces = cells) and `Nv`. Face f's ring is recovered by walking the table and appending `E_srce[k]` for every k with `E_face[k] == f`, *in table order*. That ordering *is* the geometry: `rings_from_flat_3d` never reads `E_trgt`, never sorts and never validates, so a sort, a compaction, or a scatter-into-reservoir by any future writer produces garbage rings in silence.

Before this promotion the table was a private attribute one operator invented — `grep -rn "_mesh" src/plexus/` returned nothing at all — while roughly forty consumers read it: the chemistry, the mechanics, the T1s, the cross-scale contact operators, the recorder, the salvage, the instrument's fingerprint, the renderers. A spec could schedule `cell_mechanics` without `mesh_seed` and die of `AttributeError` from inside an autograd pass.

It is now declared and allocated:

`sets: {vertex: {mesh: half_edge, cell_set: cell}}`	the declaration
`plexus.models.mesh.MeshTable`	the table, a `dict` subclass
`plexus.models.topology`	its algorithms (rings, edge split, face division)
`engine._build_mesh`	allocates an *empty* table in `build`'s pass 1
`seed_mesh`	*fills* it, rather than rebinding a new one

Why it subclasses `dict`, and this is not a style choice. `discovery_okuda/instrument.py:43` reads `if isinstance(m, dict):`. That is the acted ledger — the thing that decides whether an operator *did anything* — wrapped in a bare `except Exception: return ()`. A `MutableMapping`

that is not a `dict` subclass fails that test silently, the fingerprint collapses to `()`, and every operator whose only effect is on the mesh (`cell_grow` writing `V0f`, `cell_divide` changing `nF`, `junction_myosin` writing `myo`) scores zero acts. The campaign then stamps `valid_evidence: False` on runs that were fine.

And the namespace stays open, deliberately. Five keys are reserved; everything else is an extension namespace, because three live mechanisms depend on a name being *absent*: `cell_shape_probe` publishes a field and then deletes it (absent means "no cell dies"; zero-filled would mean "kill everything"), `ecm_gate_growth`'s entire entry condition is `'mg_scale' in m`, and the apoptosis discriminator resolves whatever name its spec gives it.

The carry was written four times. Permuting every per-face array through `keep` is one operation, and it existed in `cell_divide`, in `cell_die`, in `_carry_face_state` and in `edge_flip` — and the fourth copy silently omitted the medioapical myosin, because a per-face state added by one operator has to be known to every topology operator and that one did not get the memo. It is now `MeshTable.reindex_faces`, and the renumber of the paired *cell* set is `Hierarchy.renumber_set`, which moves all four stores the engine keeps per set. The fourth, `_delta_blocks`, is the one both hand-rolled versions missed: they guarded on `isinstance(key, tuple)` while `add_delta` keys by level *name*. It is inert today only because `chem` happens to be the cell set's coordinate block, so its deltas travel in `_delta`; a second `first_order` block on that set makes it live.

2.2 The recorder keeps the topology

`topo_record` appended its history *inside* the mesh table (`m.setdefault("hist", [])`), so the structure the engine now owns grew without bound for the length of the run and every consumer walked past a list of every frame it had ever had. The engine records it: one `MeshTable.snapshot()` per recorded row, written to both zarr and npz as a ragged store with *no pickle* — the three half-edge columns concatenated, plus row offsets, plus `nF`/`Nv` per row. Euler now computes off the core's own trajectory file,

$$V - E + F \;=\; \texttt{Nv}[t] \;-\; \tfrac{1}{2}\big(\texttt{off}[t{+}1] - \texttt{off}[t]\big) \;+\; \texttt{nF}[t] \;=\; 2,$$

which is what gate 00's closed-form row needs and what `topo_record` could not give it.

2.3 Two bugs that destroyed finished work

- `Group.create_dataset(name, data=...)` is the zarr-2 spelling. zarr 3 kept the method, deprecated it, and made `shape` a *required* keyword, so every core generation on the cluster (zarr 3.1.5) died *after* the whole simulation had run and *before* the npz was written — 25 minutes of GPU, an empty `simulation.zarr`, no trajectory. The devcontainer has zarr 2.18.7, which is why it never appeared in development.
- The npz writer put `d["pos"]` in unconditionally. A mesh model's `cell` set has no `pos` block at all — it carries `chem`, `cen`, `area` — so `None` went in as a 0-d *object* array and every read of the file died on `allow_pickle`. Worse, the recorded blocks were not written at all: a reaction–diffusion run wrote a trajectory with no reaction in it.

2.4 325 of 461 specs did not load

Not "would eventually fail": `ValueError: operator 'seed_mesh' not in registry`, for every `r0*` the campaign had ever written. The specs had been migrated to the post-seed-refactor `seed_<noun>` convention — the one core already uses for `seed_from_segmentation` — and the okuda classes had not, so the corpus split 324/137 across two spellings of one operator and the larger half was dead. Both spellings resolve now: canonical `seed_mesh` and `seed_cell_chem`, aliases `mesh_seed` and `cell_chem_seed`. 460 of 461 load; the one that does not names `morphogen_growth_3d`, which is stale in a different way and is recorded rather than guessed at.

3. Phases A–D — the operators

`tools/group_operators.py` moved them, and the choice to write a script rather than edit by hand is the reason bit-equality is a reachable claim across 3,800 lines. It hoists each source's imports, rewrites the ones that named a sibling by bare module name, and concatenates the bodies under a banner. Nothing inside a function body is touched. `-verify` compares *text*: every top-level definition's exact source, decorators included, searched for as a substring of the module it landed in.

That verify earned itself on the first run. `ClassDef.lineno` is the line of the `class` keyword, not of the `@register_operator(...)` above it, so taking it as the start of the body dropped the decorator off the *first* class in every file: `cell_geometry` silently left the registry and 461 specs stopped loading. The counting check — "70 names, looks right" — had passed.

from	to	lines
`topology_ops.py`	`src/plexus/models/topology.py`	218
`mesh_ops + t1_ops + monolayer_ops`	`operators/vertex_ops.py`	2183
`chem_ops + shape_chem_ops + shape_probe_ops`	`operators/diffusion_reaction.py`	1490
`junction_ops + medioapical_ops`	`operators/junction_ops.py`	856
`ecm_ops + block_ops`	`operators/ecm_ops.py`	864
`membrane_ops + integrin_ops`	`operators/membrane_ops.py`	2187
`mesh_contact + bm_sense + plate + surface + load`	`operators/contact_ops.py`	1183

The shims are load-bearing. Thirty files import `mesh_ops` / `chem_ops` / `topology_ops` by bare module name — `run_one.py`, `instrument.py`, `vtk_render.py`, `metrics.py` and twenty archive and analysis scripts — and the campaign is still running against them. Each moved file is a re-export, *private names included*: `t1_ops` calls `_carry_face_state` and `analyze_forces` reaches for `_engine_owns_clock`.

Two operators did not come. `mpm_boundary` and `bm_strain` are rejected by `AUDIT.md`, and a rejection that lives only in a markdown file is one the next reader re-promotes by accident. Their classes are *cut out* of the module that moved and left in `discovery_okuda/ops/membrane_ops.py` under a banner saying why: an archived spec that names one still runs, and no spec can reach either from the core registry. `mpm_boundary` overwrites grid-node velocity, so the constraint is kinematic — momentum not conserved, the reaction discarded — and its standoff is set by the B-spline stencil width, measured across `recover` 0/2/6/20 as 46.6%/3.8%/11.5%/13.9% of the sheet inside the tissue against standoffs $+0.0006/+0.0124/+0.0088/+0.0069$, never reaching the $0 \to +0.002$ that would mean "just touching". `integrin_track` is the constraint it should have been. `bm_strain` is, in AUDIT's words, "not a mechanism".

3.1 Redundancy: the four merge candidates, settled by reading them

- `interface_tension` + `interface_push` — **do not merge.** They were one operator, `rd_interface_tension`, carrying $K_{\text{purse}} \sum_{\text{iface}} \ell_e \;-\; K_{\text{extrude}} \sum_{\text{red}} a\, r$. The first term is ordinary vertex-model physics; the second does not model a force, it pays the tissue to produce the morphology the campaign was searching for. One name over both cost four rounds: `K_extrude` measured 0.0 in all 78 specs that ever carried the operator, so nothing was ever forced, and the Grounder still reported a round as "the same extrude-forced star for a fourth round" on three runs whose specs contain no such operator. The split *is* the fix.
- `ecm_stress` / `block_stress` — **merge, in Phase D.** Same body. They differ in two defaults — `scale` 0.15 for the matrix against 0.004 for the block, which is right, since the block is $\sim$130$\times$ stiffer — and in *which module-level history list* they append to. That list is the only reason the duplicate exists.

- `ecm_seed` / `block_seed` — **do not merge.** Box-minus-a-cavity with aligned fibres against two slabs of jittered lattice. Same family, different geometry.
- `integrin_adhesion` **vs** `integrin_pull/seed/track` — **do not merge.** One is membrane→epithelium, the other matrix→membrane. The shared prefix is the problem, not the code; they go in one module with that sentence at the top of it.

3.2 The inventory

module	operator	family	kind	set	implementations / alias
agent_ops.py	active_force	mechanics	exchange	particle	alias pulse_to_contraction
agent_ops.py	active_stress	mechanics	exchange	particle	alias pulse_to_active_stress
agent_ops.py	agent_divide	population	structural	cell	
agent_ops.py	agent_gather	coupling	exchange	cell	alias mpm_to_agent
agent_ops.py	agent_grow	population	structural	cell	
agent_ops.py	agent_remodel	coupling	exchange	cell	
agent_ops.py	agent_scatter	coupling	exchange	cell	alias agent_to_mpm
agent_ops.py	aggregate	hierarchy	aggregate	cell	
agent_ops.py	broadcast	hierarchy	broadcast	particle	
agent_ops.py	polarity_align	polarity	exchange	cell	alias heading_align
agent_ops.py	polarity_flow_align	polarity	exchange	cell	alias flow_align
agent_ops.py	seed_from_segmentation	seed	seed	particle	
contact_ops.py	bm_sense	signalling	structural	vertex	
contact_ops.py	ecm_gate_growth	population	structural	vertex	
contact_ops.py	ecm_load	mechanics	structural	vertex	
contact_ops.py	mesh_contact	boundary	lateral	particle	
contact_ops.py	mesh_inside	hierarchy	lateral	particle	
contact_ops.py	plate_confine	boundary	structural	vertex	
contact_ops.py	surface_track	hierarchy	structural	particle	
diffusion_reaction.py	cell_chem_diffuse	fields	lateral	cell	graph_laplacian, interface_weighted
diffusion_reaction.py	cell_chem_from_shape	fields	lateral	cell	apical_area, curvature, pressure, tension
diffusion_reaction.py	cell_chem_react	fields	lateral	cell	brusselator, gierer_meinhardt, gray_scott
diffusion_reaction.py	cell_geometry	hierarchy	aggregate	cell	
diffusion_reaction.py	cell_grow	population	structural	vertex	balance, default, sizer, timer
diffusion_reaction.py	cell_neighbours	topology	rewire	cell	
diffusion_reaction.py	cell_shape_probe	hierarchy	lateral	cell	aspect, shape_index
diffusion_reaction.py	interface_push	mechanics	lateral	vertex	
diffusion_reaction.py	interface_tension	mechanics	lateral	vertex	
diffusion_reaction.py	seed_cell_chem	seed	seed	cell	alias cell_chem_seed
ecm_ops.py	block_seed	seed	seed	particle	
ecm_ops.py	block_stress	hierarchy	lateral	particle	
ecm_ops.py	cell_exclude	boundary	structural	particle	
ecm_ops.py	ecm_from_cell	mechanics	lateral	particle	replay, sphere
ecm_ops.py	ecm_seed	seed	seed	particle	
ecm_ops.py	ecm_stress	hierarchy	lateral	particle	
field_ops.py	activation_pulse	fields	field	field	
field_ops.py	chemotax	fields	exchange	particle	
field_ops.py	decay	fields	field	field	
field_ops.py	deposit	fields	exchange	cell	
field_ops.py	diffuse	fields	field	field	finite_difference, spectral
field_ops.py	pacemaker	fields	field	field	
field_ops.py	playback	harness	field	field	
field_ops.py	sense	signalling	exchange	cell	
field_ops.py	signal	signalling	lateral	neuron	
interaction_ops.py	attraction_repulsion	interaction	lateral	particle	
interaction_ops.py	cohesion	interaction	lateral	particle	
interaction_ops.py	radius_graph	topology	rewire	particle	
interaction_ops.py	separation	interaction	lateral	particle	
interaction_ops.py	squared_law	interaction	lateral	particle	
interaction_ops.py	stillinger_weber	interaction	lateral	particle	autograd
interaction_ops.py	velocity_align	interaction	lateral	particle	alias alignment
junction_ops.py	cytokinetic_ring	mechanics	structural	vertex	

module	operator	family	kind	set	implementations / alias
junction_ops.py	junction_myosin	mechanics	structural	vertex	default, two_pool
junction_ops.py	junction_sync	mechanics	rewire	vertex	
junction_ops.py	medioapical_myosin	mechanics	lateral	cell	
membrane_ops.py	adhesion_pull	mechanics	exchange	particle	
membrane_ops.py	adhesion_seed	seed	seed	particle	
membrane_ops.py	adhesion_turnover	topology	rewire	particle	
membrane_ops.py	bm_bond	mechanics	lateral	particle	
membrane_ops.py	bm_contact	boundary	lateral	particle	
membrane_ops.py	bm_crosslink	topology	rewire	particle	
membrane_ops.py	bm_remodel	population	lateral	particle	
membrane_ops.py	bm_repel	boundary	lateral	particle	
membrane_ops.py	bm_secrete	population	structural	particle	
membrane_ops.py	bm_seed	seed	seed	particle	
okuda membrane_ops.py	bm_strain	mpm	lateral	particle	
membrane_ops.py	bm_unbond	topology	rewire	particle	
membrane_ops.py	integrin_adhesion	mechanics	lateral	particle	
membrane_ops.py	integrin_pull	mechanics	lateral	particle	
membrane_ops.py	integrin_seed	seed	structural	particle	
membrane_ops.py	integrin_track	mechanics	structural	particle	
okuda membrane_ops.py	mpm_boundary	boundary	field	field	
motion_ops.py	attractor_flow	motion	lateral	particle	
motion_ops.py	bounce	boundary	lateral	cell	
motion_ops.py	drag	motion	lateral	particle	
motion_ops.py	glide	motion	lateral	cell	
motion_ops.py	gravity	mechanics	lateral	cell	
motion_ops.py	sediment	motion	lateral	cell	
motion_ops.py	velocity_cruise	motion	lateral	particle	alias cruise
mpm_ops.py	apply_material_map	mpm	exchange	particle	
mpm_ops.py	mls_mpm_mechanics	mpm	exchange	particle	
mpm_ops.py	mpm_anchor	mechanics	lateral	particle	
mpm_ops.py	mpm_gather	mpm	exchange	particle	alias g2p
mpm_ops.py	mpm_grid_update	mpm	field	field	
mpm_ops.py	mpm_scatter	mpm	exchange	particle	alias p2g
mpm_ops.py	mpm_spin	mechanics	lateral	particle	
mpm_ops.py	mpm_strain	mpm	lateral	particle	
vertex_ops.py	cell_die	population	structural	vertex	
vertex_ops.py	cell_divide	population	structural	vertex	default, doubler, timer
vertex_ops.py	cell_mechanics	mechanics	lateral	vertex	default, monolayer
vertex_ops.py	edge_flip	topology	rewire	vertex	
vertex_ops.py	seed_mesh	seed	seed	vertex	alias mesh_seed
vertex_ops.py	topo_record	harness	structural	vertex	

91 of 93 canonical operator names resolve into `src/plexus/`. The remainder are the two `AUDIT.md` rejects, registered in `discovery_okuda` only.

4. The twin-run gate

The protocol, stated exactly. *Run the spec in okuda first, and run the twin spec in the codebase. Do not use previous okuda results — they can be updated. Run both on the L4 cluster node, in parallel.* Nothing already sitting in `log/okuda` is ever the reference: those artefacts were produced by whatever okuda was on the day they were written, and `round.py` runs campaigns into that tree while `staged.py` writes into it by hand. A promotion checked against a stale file proves that the file has not changed, which is not the question.

What counts as identical. The two writers differ — okuda's `RunArchive` writes `traj.npz` (`pos_i`, `act_i`), the core writes `trajectory.npz` (`<set>__pos`, `<set>__occ`, `<set>__chem`) — so a file hash cannot match and is not the test. What makes the two comparable at all is that okuda's own frames come out of `engine._assemble`: `run_one.py:593` reads `out["sets"]["vertex"]["pos"]` and `out["sets"]["cell"]["state"]["chem"]`, the same structure the core writes. So okuda's `pos_i` *is* the core's `vertex__pos[t]` cropped to the live prefix, and `act_i` *is* `cell__chem[t][:nF,0]`. okuda keeps the ∼60 frames its movie draws and stores their row indices in `ticks`; the core keeps every row; the okuda side is read first and its `ticks` select the core's rows.

The repeatability floor is zero, and it was measured. Byte-identity is only the right criterion if the platform can deliver it, and over 1,800 frames that is not obvious: one differing mantissa bit can move an `edge_flip` decision ($\ell < \texttt{l\_th\_frac} \cdot \bar{\ell}$) and from then on the two runs have *different topology*, at which point "max $|\Delta|$" is not small, it is meaningless. So `-phase R` runs the identical spec, the identical code and the identical commit *twice*, as two separate jobs. `b_star` at 1,800 frames, growing 2,000 cells to 12,272 through division and T1: `beb6fb24fe86dcde` both times, $\max|\Delta| = 0.0$ exactly, no first differing bit, over 120 arrays per run. The floor is zero, so `tol = 0` is a real criterion at full length and every row keeps it. A tolerance would have to be justified by a floor above zero; there is not one.

Determinism is asserted, not assumed. `engine.py` sets `torch.use_deterministic_algorithms(True, warn_only=True)`, and `warn_only` silently downgrades any kernel with no deterministic implementation — which is exactly where two runs of one spec stop matching. Each side runs with `PLEXUS_STRICT_DETERMINISM=1`, which turns the downgrade into an exception, so a side that dies of it *fails* rather than passing on a comparison it never earned.

phase	spec	frames	A	B	digest	outcome
0	`apop_many`	600	okuda@HEAD	okuda	`0c0b38ec027ca47d`	**identical**
0	`apop_one`	600	okuda@HEAD	okuda	`568eaeab77720645`	**identical**
0	`apop_patch_big`	600	okuda@HEAD	okuda	`ac5b5b85075cbf55`	**identical**
0	`apop_rings9`	600	okuda@HEAD	okuda	`cbf719d02511d058`	**identical**
0	`apopgeo_half`	600	okuda@HEAD	okuda	`1f9d217766575360`	**identical**
0	`b_null_plain`	1800	okuda@HEAD	okuda	`14aab859c575fe25`	**identical**
B-core	`b_gs_plain_soft_lo`	1800	okuda	core	`906afc1a3931063a`	**identical**
B-core	`b_star`	1800	okuda	core	`beb6fb24fe86dcde`	**identical**
R	`b_null_plain`	1800	okuda	okuda	`14aab859c575fe25`	**identical**
R	`b_star`	1800	okuda	okuda	`beb6fb24fe86dcde`	**identical**

10 of 10 rows identical.
Every row runs at its spec's own length. 100 frames was too short to test anything structural: `b_star`'s topology keeps changing past frame 800, so a 100-frame gate green-lights a promotion that has not run one of the divisions and flips it claims to preserve.

Which row is the actual claim. The `okuda@HEAD` vs `okuda` rows compare okuda before the change to okuda after it: they prove the *move* changed no bytes, which is necessary and is not the question. The `B-core` rows are the claim — `Plexus_Main.py -o generate` against `run_one.py`, different runner, different recorder, different writer, the core registry with no okuda import on one side. `b_star`'s `B-core` digest is the same hex the repeatability pair produced, so the core run, the okuda run and two independent repeats of the okuda run all agree bit for bit, positions and chemistry alike.

Four of the rows are the minisite's death scenes , run to their own length rather than truncated, because a gate whose archive stops before the deformation cannot show what it claims to test. A 45° cap of 293 cells draws the surface *inward* — an invagination, the one deformation growth cannot make; nine bands of 895 deaths close over every gap and the vesicle stays a sphere at reduced volume 0.981; 285 of 400 cells above the equator leave an ellipsoid pulled from a sphere at `gyr_prolate` 1.869, with the topology intact. `apop_many` is the only new spec in the set, and it exists because one row moved is not the hard case for a renumber: its 200 deaths are spread along the Fibonacci spiral by a stride of 10, so `keep` is interleaved rather than a contiguous block — the case a renumber written as a truncation passes and one written as a gather has to get right.

5. The okuda_ECM gates 00–04

What they were. Not assert-scripts. `test_00_spheroid.py` writes a `spec.yaml` saying what it did, a `metrics.json` and some plots — it *reports*. Its "spec" is built imperatively in Python, and its output lands wherever the script chose under `log/okuda_ECM/`. And not one of the 461 specs in `config/okuda/` names a single junction, ECM, membrane or contact operator, so 34 of the operators just promoted have no declared spec at all.

What they become. Two dedicated homes, and a threshold that exists before the run.

- `config/gates/` — one yaml per gate, lifted out of the script, carrying the sets, the operators, the schedule, the frame count, the seed, *and* the declared thresholds with their tier. The spec becomes the gate's definition rather than a side effect of running it, which is what makes a threshold auditable before the run.
- `log/gates/<gate>/` — the promoted run's outputs: `metrics.json`, the gate table as written, the plots, the movie. `log/okuda_ECM/` stays as the okuda side and is not written over, so the two can always be diffed.

The tiers are the paper's, and `basis` is the column that keeps them honest. *Bookkeeping* asks whether the code does what the operator says. *Closed form* asks whether the implementation reproduces the physics it was *given*. *Measurement* asks whether the model agrees with something observed in cells. A threshold copied off a previous run of the same code is none of those three — it is a *regression pin* — and left in the measurement tier it reads as biology. So every row also declares `basis: identity | analytic | reference | literature`, and the fraction at the foot of the table is reported against both.

And the measurement tier turns out to be available. `general.units:` is declared in `schema.py`, parsed through `plexus/units.py`, and its calibration is already on disk: `log/okuda_ECM/0u_units/units` gives one tissue length unit = 10 μm and one frame = 600 s. So gate 00 carries three rows in the unit of the phenomenon rather than of the mesh: a 13.08 h cell cycle against the 12–24 h a cultured epithelial cell takes; a 7.66 μm cell against 5–15; a 359 μm spheroid against the 100–500 μm at which a real one's core goes hypoxic and this model, which has no nutrient field, stops being about anything. Preflight *refuses* a physical row on a spec with no `units:` block, and refuses any `unit:` string that names the mesh — "0.82 grid cells" sounds small and is 15 μm.

Gate 00 was reconstructed, and that was the hard part. `test_00_spheroid.py` does not run a tissue, it *loads* one, so the model had to be recovered from the cache's provenance. Two candidate parents exist and they are *different models*: the pre-basis `cellfix_B_new.yaml` says `rate: 0.003457` and is what the cache was built from, while the `_archive_runs` copy `tissue.py` falls back to today says `rate: 0.03` and reaches 1,451 cells by frame 60 against the cache's 227. Gating against the wrong parent would fail a correct run. The reconstruction is confirmed by the run itself: 6,914 cells exactly, apical-radius fold 3.86064 against the cache's 3.8607, aspect 1.01931 against 1.0193.

Its first run failed five rows and four of them were the measuring code , which is what a gate is for. `apical_radius` was a mean where the reference is a median; `aspect_ratio` was a gyration tensor where the reference is a percentile pair; `mean_cell_diameter` fanned its triangles from the body's centroid instead of each face's and read 50.9 μm for a cell that is 7.66; and `t1_total` reconstructed the count as "new edges minus three per new face", which is wrong because a division *splits* two edges — five pairs appear and two vanish — so it read 16,169 against 1,499. Excluding divisions by geometry instead of by arithmetic gave 2,890, still not 1,499, because a 3D reversible network reconnection rewires more than the one edge it is named for. Two numbers each right about a different quantity is the worst kind of disagreement: neither side is wrong and the row is

meaningless. So the engine now records the operators' own counters — `MeshTable.SCALAR_RECORD`: `n_t1`, `n_apop`, `div_blocked`, `apop_spill` — and the row reads 1,499.

And gate 01's did the same thing twice, which is the point of having them. Myosin is the hard case for this promotion because it is the only state in the model that lives on a *relation* rather than on a body: its array is indexed by half-edge, it is written before the frame's topology operators rewire that index, and `junction_sync` has to re-key it afterwards. On the archived 401-frame run, 56 of 200 snapshots carried a myosin array 6 to 1,356 entries short of the edge arrays — and every reader indexes it positionally, so each of those frames coloured, averaged and thresholded the wrong junctions. The gate asserts the alignment on the belted arm (0) and reports it on the arm with the re-keying removed (2,106), so the check has a positive control rather than only a green light. `junction_sync` itself is asserted *trajectory-neutral at exactly zero* — not "small" — because it writes only `m["myo"]` and nothing reads it before the tick ends; the measured maximum vertex displacement between the two arms is 0.0.

Two of its rows failed on the first grading and both were the threshold. `t1_rate` had been derived by dividing 1,499 exchanges by the *final* cell count instead of the mean over the run — the tissue spends most of its frames small — and read 0.00215 against a demand for 0.00105; the archive says 0.002206, so the measurement was right to 2% and the arithmetic behind the threshold was not. And `hot_junction_fraction` asked for at least 2% of junctions above 1.5× the mean while its own sibling asserted $p_{98}/\text{mean} \geq 1.20$: those are not two checks, they are a contradiction, since $p_{98} = 1.42$ means exactly 2% lie above 1.42 and so at most 2% can lie above 1.5. A row that cannot pass while its neighbour passes is not strict, it is wrong. It was replaced by one that is independent — the correlation between a junction's myosin and its length, which separates a length-keyed belt from a tension-keyed one where the dispersion cannot, because both keyings raise the dispersion identically.

And that replacement failed too, with the sign backwards. I asserted $r < -0.10$, reasoning that a belt contracts short junctions. It measured +0.639. The operator's own comment settles it: *"length-keyed, +: longer → more myosin → shorter. Negative feedback on length"* — the target is $a\,(1 + \beta(\ell/\bar{\ell} - 1))$, linear and increasing in ℓ, so the correlation must be positive, and it is below +1 only because the myosin relaxes toward that target over 20 frames. The same comment warns that "getting this backwards silently converts the experiment into its own control, so it is written out rather than inferred", which is precisely what a threshold reasoned from intuition rather than from the update rule did here. The row is `basis: analytic` now, because the sign follows from the rule and needs no previous run.

gate	row	tier	basis	declared	measure
00_spheroid	seed_cell_count	bookkeeping	identity	eq 200	20
00_spheroid	cell_count_never_falls	bookkeeping	identity	ge 0	
00_spheroid	occupancy_matches_topology	bookkeeping	identity	eq 0	
00_spheroid	topology_delta_ledger	bookkeeping	identity	eq 0	
00_spheroid	no_nan	bookkeeping	identity	eq 0	
00_spheroid	reservoir_headroom	bookkeeping	identity	le 0.95	0.54033
00_spheroid	euler_characteristic	closed form	analytic	eq 2	
00_spheroid	vertex_valence	closed form	analytic	eq 1.0	
00_spheroid	mean_neighbours_per_cell	closed form	analytic	lt 1e-09	
00_spheroid	final_cell_count	measurement	reference	eq 6914	691
00_spheroid	apical_radius_fold	measurement	reference	within [3.8607, 0.002]	3.8606
00_spheroid	aspect_ratio	measurement	reference	within [1.0193, 0.01]	1.0193
00_spheroid	t1_total	measurement	reference	eq 1499	149
00_spheroid	t1_rail_fraction	measurement	reference	le 0.4	0.36904
00_spheroid	cell_cycle_hours	measurement	literature	interval [12.0, 24.0]	13.075
00_spheroid	mean_cell_diameter_um	measurement	literature	interval [5.0, 15.0]	7.6607
00_spheroid	spheroid_diameter_um	measurement	literature	interval [100.0, 500.0]	359.3
01_junction	myosin_array_aligned_with_half_edges	bookkeeping	identity	eq 0	
01_junction	myosin_unaligned_without_sync	bookkeeping	identity	ge 0	210
01_junction	cell_count_never_falls	bookkeeping	identity	ge 0	

gate	row	tier	basis	declared	measure
01_junction	euler_characteristic	closed form	analytic	eq 2	
01_junction	junction_sync_is_trajectory_neutral	closed form	analytic	eq 0.0	
01_junction	t1_total	measurement	reference	eq 1499	149
01_junction	t1_rate_per_cell_per_frame	measurement	reference	within [0.0022, 0.0004]	0.0021541
01_junction	mean_junction_length_fold	measurement	reference	within [0.63, 0.08]	0.62938
01_junction	myosin_is_localised_not_merely_present	measurement	reference	ge 1.2	1.4228
01_junction	tissue_mean_myosin_pinned_to_activity	measurement	reference	within [1.0, 0.08]	1.0157
01_junction	myosin_accumulates_on_long_junctions	measurement	analytic	gt 0.1	0.63898
01_junction	belt_suppresses_intercalation	measurement	reference	lt 0.0	-0.00050482
01_junction	junction_lifetime_hours	measurement	literature	interval [0.9, 1.0]	0.99258
02_ecm_block	particle_count	bookkeeping	identity	eq 90000	9000
02_ecm_block	no_particle_outside_the_domain	bookkeeping	identity	eq 0	
02_ecm_block	no_nan	bookkeeping	identity	eq 0	
02_ecm_block	free_fall_acceleration	closed form	analytic	within [-2.5, 0.05]	-2.4878
02_ecm_block	lowest_centroid	measurement	reference	interval [0.05, 0.3]	0.153
02_ecm_block	rebound_happens	measurement	reference	gt 0.05	0.20416
02_ecm_block	strand_length_um	measurement	literature	interval [20.0, 500.0]	105.85

18 of 37 rows are verification (bookkeeping or closed form, 49%); 13 pin a regression against a previous run of this same model; and **5 compare the model with something observed in cells** (14%). The paper reports roughly two thirds bookkeeping-or-closed-form for the `ecm` study; this table looks better than that only because `basis` separates the regression pins out of the measurement tier, where they would otherwise sit and read as biology.
Outcomes: 'PASS': 37.

6. Status

phase	what	state
0	mesh as an engine concept; recorder; determinism; live snapshot	**done**
A	`OKUDA_PROMOTION.md`: 50 names, contracts, merge verdicts	**done**
B	`vertex_ops.py`, `diffusion_reaction.py`, `models/topology.py`	**done**
B-core	the core runs the model with no okuda import, bit-identical	**done**
C	`junction_ops.py`	**done**
D	`ecm_ops.py`, `membrane_ops.py`, `contact_ops.py`	**done**
0b	MPM regrouped into `mpm_ops.py`	not started
gates 00, 01, 02	lifted into `config/gates/`, run from core, **37/37 green**	**done**
gate 04	liftable; 6 rows blocked on a 32 MB uncommitted replay	**in flight**
gate 03	not liftable: needs a seeder, four operators and massed vertex dynamics	not started
0b/E	core regrouped into ten modules; VTK is the engine's renderer	**done**
F	the three minisite scenes under `config/atlas/`	not started
G	replication: `r010_00_ctrl` and `b_star` as twin runs	not started